

Лабораторная работа № 8.

Защита REST API и валидация пользовательских данных

Тема: express-validator, Helmet, rate limiting и ограничение HTTP request

Цель: Добавить в Internet Shop API многоуровневую защиту: server-side validation, фильтрацию неизвестных полей, ограничение размера JSON, security headers и rate limiting.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте проект `express-shop`. Должны работать Express Router, Controller/Service, Prisma/PostgreSQL, JWT, `requireAuth`, `requireRole` и CORS.

Основной принцип

Любые данные из `req.body`, `req.params`, `req.query` и HTTP headers считаются недоверенными, пока backend их не проверил.

1. Установите пакеты

```
PowerShell

npm install express-validator helmet express-rate-limit
```

2. Ограничьте размер JSON body

```
app.js

app.use(express.json({
  limit: "100kb"
}));
```

Слишком большой request body должен быть отклонён до выполнения Controller и Service. Для обычных JSON-запросов Internet Shop лимита 100kb достаточно.

3. Создайте validation middleware

```
middleware/validationMiddleware.js

import {
  matchedData,
  validationResult
} from "express-validator";

export function validateRequest(req, res, next) {
  const result = validationResult(req);

  if (!result.isEmpty()) {
    return res.status(400).json({
```

```

        error: "Validation failed",
        details: result.array()
    });
}

req.validated = matchedData(req, {
    includeOptionals: true
});

next();
}

```

`validationResult()` собирает ошибки validation, а `matchedData()` извлекает только поля, для которых были выполнены validation/sanitization rules.

4. Создайте validator для Product

validators/productValidators.js

```

import {
    body,
    checkExact,
    param
} from "express-validator";

export const createProductValidation = [
    body("name")
        .isString()
        .withMessage("name must be a string")
        .bail()
        .trim()
        .isLength({ min: 2, max: 150 })
        .withMessage("name length must be 2..150"),

    body("price")
        .isInt({ min: 1, max: 100000000 })
        .withMessage("price must be a positive integer")
        .toInt(),

    body("categoryId")
        .isInt({ min: 1 })
        .withMessage("categoryId must be a positive integer")
        .toInt(),

    body("inStock")
        .optional()
        .isBoolean()
        .withMessage("inStock must be boolean")
        .toBoolean(),

    checkExact([], {
        locations: ["body"],
        message: "Unknown field"
    })
];

export const productIdValidation = [
    param("id")
        .isInt({ min: 1 })
        .withMessage("id must be a positive integer")
        .toInt()

```

```
];
```

5. Подключите validation к POST /api/products

```
routes/productRoutes.js

router.post(
  "/",
  requireAuth,
  requireRole("admin"),
  createProductValidation,
  validateRequest,
  createProduct
);
```

Порядок middleware важен: сначала Authentication и Authorization, затем validation, затем Controller.

6. Используйте только req.validated

```
controllers/productController.js

export async function createProduct(req, res, next) {
  try {
    const product = await addProduct(req.validated);

    res.status(201).json(product);
  } catch (error) {
    next(error);
  }
}
```

Не передавайте req.body напрямую

Не используйте `prisma.product.create({ data: req.body })`. Клиент может добавить неожиданные поля. Передавайте только проверенные данные.

7. Проверьте корректный Product

```
POST /api/products

POST /api/products
Authorization: Bearer <adminAccessToken>
Content-Type: application/json

{
  "name": "USB Hub",
  "price": 12000,
  "categoryId": 1,
  "inStock": true
}
```

При корректных данных должен быть создан новый Product.

8. Проверьте неправильный тип price

```
POST /api/products
Authorization: Bearer <adminAccessToken>
Content-Type: application/json

{
  "name": "USB Hub",
  "price": "free",
  "categoryId": 1
}
```

Ожидаемая идея ответа

```
{
  "error": "Validation failed",
  "details": [
    {
      "msg": "price must be a positive integer"
    }
  ]
}
```

Ожидаемый HTTP status — 400. Request не должен попасть в Product Service.

9. Проверьте диапазоны

1. Отправьте price: 0.
2. Отправьте categoryId: 0.
3. Отправьте имя из одного символа.
4. Отправьте имя длиннее 150 символов.
5. Убедитесь, что каждый некорректный request возвращает 400.

10. Проверьте неизвестное поле

```
POST /api/products
Authorization: Bearer <adminAccessToken>
Content-Type: application/json

{
  "name": "USB Hub",
  "price": 12000,
  "categoryId": 1,
  "role": "admin"
}
```

checkExact() должен добавить validation error для неизвестного поля role.

11. Подключите validation id

```
routes/productRoutes.js

router.get(
  "/:id",
  productIdValidation,
  validateRequest,
```

```
    getProductById  
  );
```

controllers/productController.js

```
export async function getProductById(req, res, next) {  
  try {  
    const product = await findProductById(  
      req.validated.id  
    );  
  
    if (!product) {  
      return res.status(404).json({  
        error: "Product not found"  
      });  
    }  
  
    res.json(product);  
  } catch (error) {  
    next(error);  
  }  
}
```

12. Проверьте неправильные id

6. Выполните GET /api/products/abc.
7. Выполните GET /api/products/0.
8. Выполните GET /api/products/-1.
9. Убедитесь, что backend возвращает 400 до Prisma query.

13. Создайте validator для Order

validators/orderValidators.js

```
import {  
  body,  
  checkExact  
} from "express-validator";  
  
export const createOrderValidation = [  
  body("items")  
    .isArray({ min: 1, max: 50 })  
    .withMessage("items must contain 1..50 positions"),  
  
  body("items.*.productId")  
    .isInt({ min: 1 })  
    .withMessage("productId must be positive integer")  
    .toInt(),  
  
  body("items.*.quantity")  
    .isInt({ min: 1, max: 100 })  
    .withMessage("quantity must be 1..100")  
    .toInt(),  
  
  checkExact([], {  
    locations: ["body"],  
    message: "Unknown field"  
  })  
]
```

```
];
```

14. Подключите Order validation

routes/orderRoutes.js

```
router.post(
  "/",
  requireAuth,
  createOrderValidation,
  validateRequest,
  createOrderController
);
```

controllers/orderController.js

```
export async function createOrderController(req, res, next) {
  try {
    const order = await createOrder(
      req.user.id,
      req.validated.items
    );

    res.status(201).json(order);
  } catch (error) {
    next(error);
  }
}
```

15. Проверьте Order validation

10. Отправьте пустой массив `items`.
11. Отправьте `quantity: 0`.
12. Отправьте `quantity: 101`.
13. Отправьте отрицательный `productId`.
14. Добавьте неизвестное поле `userId` в `body`.
15. Убедитесь, что некорректные запросы завершаются 400.

Business rules остаются в Service

Validator проверяет форму `productId` и `quantity`, но существование `Product`, доступность товара и транзакцию создания заказа по-прежнему проверяет `Order Service`.

16. Подключите Helmet

app.js

```
import helmet from "helmet";

app.use(helmet());
```

16. Перезапустите backend.

17. Выполните любой GET request.
18. Откройте Response Headers.
19. Найдите Content-Security-Policy.
20. Найдите X-Content-Type-Options.
21. Найдите Referrer-Policy.
22. Убедитесь, что X-Powered-By отсутствует.

17. Добавьте общий rate limiter

```
middleware/rateLimiters.js

import { rateLimit } from "express-rate-limit";

export const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 300,
  standardHeaders: true,
  legacyHeaders: false
});

export const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 10,
  standardHeaders: true,
  legacyHeaders: false
});
```

18. Подключите limiters

```
app.js

app.use("/api", apiLimiter);
```

```
routes/authRoutes.js

router.post(
  "/login",
  loginLimiter,
  loginValidation,
  validateRequest,
  login
);
```

19. Проверьте login limiter

Для быстрого теста временно уменьшите `limit` у `loginLimiter` до 3.

23. Отправьте несколько login requests подряд.
24. После достижения лимита проверьте HTTP status 429.
25. Посмотрите RateLimit headers в response.
26. Верните рабочее значение `limit: 10` после эксперимента.

20. Проверьте ограничение body size

Создайте JSON body больше заданного лимита и отправьте его в любой POST endpoint.

Ожидаемый HTTP status — 413. Controller не должен выполняться.

21. Добавьте безопасный error handler

```
middleware/errorHandler.js

export function errorHandler(error, req, res, next) {
  console.error(error);

  if (error.type === "entity.too.large") {
    return res.status(413).json({
      error: "Request body too large"
    });
  }

  res.status(error.status ?? 500).json({
    error: error.status && error.status < 500
      ? error.message
      : "Internal server error"
  });
}
```

Не возвращайте stack trace клиенту

Подробная ошибка нужна в server logs. В HTTP response не отправляйте stack trace, database connection strings, Prisma internals, passwords или tokens.

Самостоятельное задание

27. Добавьте validation для PATCH /api/products/:id.
28. Сделайте поля PATCH optional, но проверяйте тип и диапазон переданных значений.
29. Запретите неизвестные body fields.
30. Проверьте validation для id в DELETE route.
31. Добавьте validation email/password к login.
32. Проверьте 400 для некорректного email.
33. Проверьте 413 для oversized JSON.
34. Проверьте Helmet response headers.
35. Проверьте 429 для login limiter.
36. Убедитесь, что обычные корректные запросы продолжают работать.

Требования к результату

- Установлены express-validator, helmet и express-rate-limit.
- Создан validateRequest.
- Product body валидируется на backend.
- Order items валидируются на backend.
- Path parameters валидируются.
- Неизвестные поля отклоняются или не передаются в Service.
- Controller использует req.validated.
- JSON body имеет разумный size limit.
- Helmet подключён.
- API rate limiter подключён.
- Login использует отдельный более строгий limiter.
- Ошибка oversized body обрабатывается как 413.
- Частые requests получают 429.

- В HTTP errors не возвращаются внутренние stack traces.

Критерии оценивания

№	Критерий
1	express-validator установлен
2	validateRequest реализован
3	Product POST проходит server-side validation
4	Product id валидируется
5	Order items валидируются
6	Используется matchedData/req.validated
7	Неожиданные fields не попадают в Service
8	Некорректные данные возвращают 400
9	JSON body имеет limit
10	Oversized body возвращает 413
11	Helmet подключён
12	Security headers присутствуют
13	Общий API limiter подключён
14	Login limiter возвращает 429 при превышении лимита
15	Студент объясняет defence in depth для REST API